Created By Rahul Pal

1. Approach

The platform is containerised using Docker Compose with three isolated services: a Django REST API backend, a React frontend built with Vite, and a PostgreSQL database. Each service runs in its own container and communicates over a shared Docker bridge network.

The goal was a single-command setup — docker compose up --build — that handles migrations, static files, and service orchestration without any manual steps after the initial .env configuration.

KEY DECISIONS

- Separate Dockerfiles per service keep each image minimal and independently rebuildable.
- The backend waits for the db service to become healthy before starting, preventing connection errors on first boot.
- The frontend uses Vite's dev-server proxy to forward /api and /media requests to the backend, eliminating CORS complexity in development.
- Named Docker volumes persist PostgreSQL data and uploaded media files across container restarts.
- Environment variables are managed via python-decouple in Django, reading from a .env file that is never committed to source control.
- The custom JWT serializer embeds is_staff into the token payload, so the frontend can show or hide admin UI immediately on login without an extra API call.

2. Folder Structure

The repository is organised into two top-level application directories alongside the Docker Compose configuration file.

PATH	PURPOSE
`docker-compose.yml`	Orchestrates all three services
`event_platform/`	Django backend root
`Dockerfile`	Backend container image
`requirements.txt`	Python dependencies
`.env.example`	Environment variable template
`manage.py`	Django management entry point
`event_platform/settings.py`	Project settings (reads .env)
`event_platform/urls.py`	Root URL dispatcher
`events/models.py`	Event & Registration models
`events/serializers.py`	DRF serializers
`events/views.py`	APIView-based endpoints
`events/urls.py`	App URL patterns
`events/permissions.py`	IsAdminOrReadOnly permission
`events/token_serializers.py`	JWT + is_staff claim
`events/admin.py`	Django admin configuration
`event_frontend/`	React frontend root
`Dockerfile`	Frontend container image
`vite.config.js`	Vite build + dev proxy config
`src/App.jsx`	Router and route guards
`src/context/AuthContext.jsx`	JWT auth state (Context API)
`src/services/api.js`	Axios instance + all API calls
`src/components/Navbar.jsx`	Navigation with admin dropdown
`src/components/AdminRoute.jsx`	Admin-only route guard
`src/pages/Dashboard.jsx`	Event grid with search
`src/pages/CreateEvent.jsx`	Admin: create event form
`src/pages/ManageEvents.jsx`	Admin: manage events table
`src/pages/EditEvent.jsx`	Admin: edit event form

3. Backend Dockerfile

Located at event_platform/Dockerfile. Uses a slim Python 3.11 image, installs dependencies from requirements.txt, copies application code, and runs the Django development server on port 8000.

```
# ── Backend Dockerfile ────────────────────
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies for psycopg2 and Pillow
RUN apt-get update && apt-get install -y \
    libpq-dev gcc && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source
COPY . .

# Create media directory for uploads
RUN mkdir -p /app/media

EXPOSE 8000

# Run migrations then start the server
CMD ["sh", "-c",
     "python manage.py migrate &&
      python manage.py runserver 0.0.0.0:8000"]
```

NOTES

- libpq-dev and gcc are required to compile psycopg2-binary on Debian slim images.
- migrate runs automatically on every container start — safe because Django migrations are idempotent.
- For production, replace runserver with gunicorn event_platform.wsgi:application.

4. Frontend Dockerfile

Located at event_frontend/Dockerfile. Uses Node 18 Alpine, installs npm dependencies, and starts the Vite development server on port 3000 with host binding so it is accessible from outside the container.

```
# ── Frontend Dockerfile ────────────────────
FROM node:18-alpine

WORKDIR /app

# Install dependencies first (layer cache)
COPY package*.json .
RUN npm install

# Copy source code
COPY . .

EXPOSE 3000

# Start Vite dev server accessible on the network
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

NOTES

- package*.json is copied before the source so npm install is only re-run when dependencies change — not on every source code edit.
- --host 0.0.0.0 is required for Vite to accept connections from outside the container.
- For production, use a multi-stage build: npm run build in stage 1, serve the dist/ folder with nginx in stage 2.

5. docker-compose.yml

Defines and wires together all three services. The db service uses the official PostgreSQL 15 Alpine image. The backend depends on db being healthy. The frontend depends on the backend.

```
version: "3.9"

services:

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB:       event_platform_db
      POSTGRES_USER:     postgres
      POSTGRES_PASSWORD: yourpassword
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build: ./event_platform
    ports:
```

```
      - "8000:8000"
    env_file:
      - ./event_platform/.env
    volumes:
      - ./event_platform:/app
      - media_data:/app/media
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./event_frontend
    ports:
      - "3000:3000"
    volumes:
      - ./event_frontend:/app
      - /app/node_modules
    depends_on:
      - backend

volumes:
  postgres_data:
  media_data:
```

6. Service Overview

SERVICE	PORT	IMAGE	ROLE
db	5432	postgres:15-alpine	Primary data store. Data persisted in named volume postgres_data.
backend	8000	python:3.11-slim	Django REST API. Serves /api/* endpoints and /media/ uploads.
frontend	3000	node:18-alpine	React app. Proxies /api and /media to the backend container.

7. Running the Project

1. Copy event_platform/.env.example to event_platform/.env and fill in SECRET_KEY and DB_PASSWORD.
2. Run: docker compose up --build
3. In a second terminal, create the admin account: docker compose exec backend python manage.py createsuperuser

4 Open http://localhost:3000 for the app, http://localhost:8000/admin/ for the Django panel.

8. Environment Variables

VARIABLE	EXAMPLE VALUE	DESCRIPTION
SECRET_KEY	random-50-char-string	Django cryptographic key — keep secret
DEBUG	True	Set False in production
ALLOWED_HOSTS	localhost,127.0.0.1	Comma-separated allowed hostnames
DB_NAME	event_platform_db	PostgreSQL database name
DB_USER	postgres	Database username
DB_PASSWORD	yourpassword	Database password
DB_HOST	db	Use 'db' for Docker, 'localhost' for manual
DB_PORT	5432	PostgreSQL port
CORS_ALLOWED_ORIGINS	http://localhost:3000	Comma-separated allowed frontend origins